

P3070R2

Formatting enums

Published Proposal, 2025-01-08

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

- 1 Introduction**
- 2 Changes since R1**
- 3 Changes since R0**
- 4 SG16 Poll**
- 5 Motivation and Scope**
- 6 Proposed Change**
- 7 Impact on the Standard**
- 8 Wording**
- 9 Alternatives Considered**
- 10 Implementation**
- 11 Acknowledgements**

Appendix A: Benchmark

References

Informative References

"It is a mistake to think you can solve any major problems just with potatoes." — Douglas Adams

§ 1. Introduction

`std::format`, introduced in C++20, has significantly improved string formatting in C++. However, custom formatting of enumeration types currently requires creating `formatter` specializations which can be unnecessarily verbose for the common case of forwarding to other formatters. This proposal aims to introduce a more streamlined method for defining custom formatters for enums. When formatting enums as integers, this approach is more efficient than using a `formatter` specialization.

§ 2. Changes since R1

- Added Wording, Alternatives Considered and Acknowledgements sections.
- Clarified that `std::byte` formatting will be proposed in a separate paper.

§ 3. Changes since R0

- Included the SG16 poll results.

§ 4. SG16 Poll

Poll 3: Forward P3070R0 to LEWG.

No objection to unanimous consent.

§ 5. Motivation and Scope

Enums are fundamental in C++ for representing sets of named constants. Often, there is a need to convert these enums to string representations, particularly for logging, debugging, or interfacing with

users. The current methods for customizing enum formatting in `std::format` are not as user-friendly as they could be.

With the introduction of a `format_as` extension point for enums, we aim to:

- Simplify the process of defining custom formatting representations for enums.
- Improve enum formatting efficiency.
- Ensure compatibility with existing code and extension mechanisms.

Consider the following example:

```
namespace kevin_namespace {  
enum class film {  
    house_of_cards, american_beauty, se7en = 7  
};  
}
```

If we want to format this enum as its underlying type with `std::format` we have two options. The first option is defining a formatter specialization:

```
template <>  
struct std::formatter<kevin_namespace::film> : formatter<int> {  
    auto format(kevin_namespace::film f, format_context& ctx) const {  
        return formatter<int>::format(std::to_underlying(f), ctx);  
    }  
};
```

The drawback of this approach is that, even when forwarding to another `formatter`, it introduces a significant amount of boilerplate code. Additionally, this customization cannot be implemented within the same namespace.

The second option is converting the enum to the underlying type:

```
film f = kevin_namespace::se7en;  
auto s = std::format("{} ", std::to_underlying(f));
```

The drawback of this option is that the conversion must be performed at every call site, adding unnecessary complexity and repetition.

§ 6. Proposed Change

The current paper proposes adding a `format_as` extension point to `std::format`. `format_as` is a function discovered by argument-dependent lookup (ADL) that takes an enum to be formatted as an argument and converts it to an object of another formattable type, typically an integer or a string. It acts as a shorthand for defining a `formatter` specialization and is fully compatible with existing extension mechanisms.

This significantly improves the user experience by eliminating almost all boilerplate code:

Before:

```
namespace kevin_namespacy {
    enum class film {...};
}
template <>
struct std::formatter<kevin_namespacy::film> : formatter<int> {
    auto format(kevin_namespacy::film f, format_context& ctx) const {
        return formatter<int>::format(std::to_underlying(f), ctx);
    }
};
```

After:

```
namespace kevin_namespacy {
    enum class film {...};
    auto format_as(film f) { return std::to_underlying(f); }
}
```

The semantics of `format_as` is the same as the corresponding "forwarding" `formatter` specialization.

`format_as` can be used to format enums as strings as well:

```
enum class color {red, green, blue};

auto format_as(color c) -> std::string_view {
    switch (c) {
        case color::red:    return "red";
        case color::green:  return "green";
        case color::blue:   return "blue";
    }
}
```

```

    }
}

auto s = std::format("{} ", color::red); // s == "red"

```

Apart from usability improvements, if the target type is one of the built-in types directly supported by `std::format`, formatting can be implemented more efficiently. Instead of going through the general-purpose `formatter` API the enum can be converted directly to the built-in type at the call site. And conversion from an enum to its underlying type is effectively a noop so there is no effect on the binary size.

The difference in performance can be seen in the following benchmark results for an enum similar to `std::byte`:

Benchmark	Time	CPU	Iterations
BM_Formatter	17.7 ns	17.7 ns	38037070
BM_FormatAs	8.90 ns	8.88 ns	79036210

This will allow making `std::byte` formattable with ~2x better performance than using a `formatter` specialization. It will be done in a follow-up paper.

This can be trivially extended to other user-defined types and not just enums. At the time of writing of R0 of this paper we only had extensive usage experience with enums but since the paper was in the review pipeline for a long time we now have implementation and usage experience with all types.

§ 7. Impact on the Standard

This proposal is an additive change to the existing `<format>` standard library component and does not require changes to current language features or core library interfaces. It is a backward-compatible enhancement that addresses a common use case in `std::format`.

§ 8. Wording

Modify [\[format.arg\]](#):

```
template<class T> explicit basic_format_arg(T& v) noexcept;
```

Constraints: `T` satisfies *formattable-with*<`Context`>.

Preconditions: If `decay_t<T>` is `char_type*` or `const char_type*`, `static_cast<const char_type*>(v)` points to a NTCTS ([\[defns.ntcts\]](#)).

Effects: Let `TD` be `remove_const_t<T>`.

If `format_as(v)` is a valid expression and `TD` is an enumeration type, let `u` be `format_as(v)` and `U` be `remove_cvref_t<decltype(u)>`. Otherwise, let `u` be `v` and `U` be `TD`.

- If `TDU` is `bool` or `char_type`, initializes `value` with `vu`;
- otherwise, if `TDU` is `char` and `char_type` is `wchar_t`, initializes `value` with `static_cast<wchar_t>(static_cast<unsigned char>(vu))`;
- otherwise, if `TDU` is a signed integer type ([\[basic.fundamental\]](#)) and `sizeof(TDU) <= sizeof(int)`, initializes `value` with `static_cast(vu)`;
- otherwise, if `TDU` is an unsigned integer type and `sizeof(TDU) <= sizeof(unsigned int)`, initializes `value` with `static_cast<unsigned int>(vu)`;
- otherwise, if `TDU` is a signed integer type and `sizeof(TDU) <= sizeof(long long int)`, initializes `value` with `static_cast<long long int>(vu)`;
- otherwise, if `TDU` is an unsigned integer type and `sizeof(TDU) <= sizeof(unsigned long long int)`, initializes `value` with `static_cast<unsigned long long int>(vu)`;
- otherwise, if `TDU` is a standard floating-point type, initializes `value` with `vu`;
- otherwise, if `TDU` is a specialization of `basic_string_view` or `basic_string` and, `TDU::value_type` is `char_type` and `format_as(v)` is not a valid expression, initializes `value` with `basic_string_view<char_type>(v.data(), v.size())`;
- otherwise, if `decay_t<TDU>` is `char_type*` or `const char_type*` and `format_as(v)` is not a valid expression, initializes `value` with `static_cast<const char_type*>(v)`;
- otherwise, if `is_void_v<remove_pointer_t<TDU>>` is true or `is_null_pointer_v<TD>` is true, initializes `value` with `static_cast<const void*>(vu)`;
- otherwise, initializes `value` with `handle(v)`.

Modify [\[format.formatter.spec\]](#):

The functions defined in [\[format.functions\]](#) use specializations of the class template `formatter` to format individual arguments.

Let `charT` be either `char` or `wchar_t`. Each specialization of `formatter` is either enabled or disabled, as described below. A *debug-enabled* specialization of `formatter` additionally provides a public, constexpr, non-static member function `set_debug_format()` which modifies the state of the `formatter` to be as if the type of the *std-format-spec* parsed by the last call to `parse` were `?`. Each header that declares the template `formatter` provides the following enabled specializations:

- The debug-enabled specializations

```
template<> struct formatter<char, char>;
template<> struct formatter<char, wchar_t>;
template<> struct formatter<wchar_t, wchar_t>;
```

...

The `parse` member functions of these formatters interpret the format specification as a *std-format-spec* as described in [\[format.string.std\]](#).

Let *format-as-type*<`T`> for type `T` be `remove_cvref_t<decltype(format_as(declval<const T&>()))>`. Each header that declares the template `formatter` provides the following enabled specialization:

```
template<class T, class charT>
    requires (formattable<format-as-type<T>, charT> && is_enum_v<T>)
    struct formatter<T, charT> {
    private:
        formatter<format-as-type<T>, charT> fmt_; // exposition-only

    public:
        constexpr format_parse_context::iterator parse(format_parse_context& ctx
```

```
template <typename FormatContext>
    typename FormatContext::iterator format(const T& val, FormatContext& ctx)
};
```

```
constexpr format_parse_context::iterator parse(format_parse_context& ctx);
```

Returns: `fmt_.parse(ctx)`.

```
template <typename FormatContext>
    typename FormatContext::iterator format(const T& val, FormatContext& ctx)
```

Returns: `fmt_.format(format_as(val), ctx)`.

§ 9. Alternatives Considered

Another option is to use the `format_kind` extension point:

```
namespace kevin_namespacy {
    enum class film {
        house_of_cards, american_beauty, se7en = 7
    };
}

template<>
std::format_as std::format_kind<kevin_namespacy::film> =
    [](kevin_namespacy::film f) {
        return std::to_underlying(f);
    };
```

where `format_as` is

```
template <typename F>
struct format_as {
    F f;

    template<typename T>
    constexpr decltype(auto) operator()(T&& t) const {
```



```
        return std::invoke(f, std::forward<T>(t));  
    }  
};
```

The benefit of this approach is that it eliminates the need to introduce another extension point. However, it has several drawbacks compared to the ADL-based solution:

- It's more cumbersome to use.
- It introduces an extra level of indirection, and at least the debug code will be less efficient.
- It doesn't allow specifying a single conversion for all enums in a namespace which, based on usage experience in {fmt}, is an important use case.

For these reasons, this approach is not proposed in the current paper.

§ 10. Implementation

The proposed extension API has been implemented in the open-source {fmt} library ([\[FMT\]](#)) and as of January 2025 has been shipping for three major versions for enums and two major version for other user-defined types.

§ 11. Acknowledgements

Thanks to Tomasz Kamiński for providing useful feedback and suggesting an alternative extension API.

§ Appendix A: Benchmark

This appendix gives the source code of the benchmark used for comparing performance of `format_as` with a `formatter` specialization.

```
#include <benchmark/benchmark.h>  
#include <fmt/core.h>  
  
enum class byte_for_formatter : unsigned char {};  
  
template <>  
struct fmt::formatter<byte_for_formatter> : fmt::formatter<unsigned char>
```

```

    auto format(byte_for_formatter b, fmt::format_context& ctx) {
        return fmt::formatter<unsigned char>::format(
            static_cast<unsigned char>(b), ctx);
    }
};

enum class byte_for_format_as : unsigned char {};

auto format_as(byte_for_format_as b) { return static_cast<unsigned char>(b); }

static void BM_Formatter(benchmark::State& state) {
    auto b = byte_for_formatter();
    for (auto _ : state) {
        std::string formatted = fmt::format("{} ", b);
        benchmark::DoNotOptimize(formatted);
    }
}
BENCHMARK(BM_Formatter);

static void BM_FormatAs(benchmark::State& state) {
    auto b = byte_for_format_as();
    for (auto _ : state) {
        std::string formatted = fmt::format("{} ", b);
        benchmark::DoNotOptimize(formatted);
    }
}
BENCHMARK(BM_FormatAs);

BENCHMARK_MAIN();

```

§ References

§ Informative References

[FMT]

Victor Zverovich; et al. *The {fmt} library*. URL: <https://github.com/fmtlib/fmt>